

PROJECT 10: ShardMaster

"Automatic Database Sharding & Rebalancing"

[Consistent hashing](#)

[Sharing and sharing keys](#)

[Rebalancing](#)

WHAT IS THIS PROJECT?

Build a sharding system that:

1. Distributes data across multiple databases
2. Routes queries to correct shard
3. Adds new shards automatically
4. Rebalances data without downtime
5. Scales to billions of records

Real Example:

1 billion users, need 4 shards:

Shard 1: users 0-249M

Shard 2: users 250M-499M

Shard 3: users 500M-749M

Shard 4: users 750M-999M

As you grow, add Shard 5

Rebalance: Some users move 1→5, 2→5, 3→5, 4→5

Zero downtime!

Users stay online entire time

V1 SCOPE: 3 MONTHS (ADVANCED!)

Core Features:

1. Shard Directory

- Know which shard has which user
- `user_id` → `shard_id` lookup ($O(1)$)
- Cached in memory for speed
- API: `GET /shard?user_id=123` → returns "shard_2"

2. Hash-Based Sharding

- `shard_id = hash(user_id) % num_shards`
- Deterministic: same user always same shard
- Simple but works

3. Routing Middleware

- Intercept queries
- Extract shard key (`user_id`)
- Route to correct shard
- Return result to app

4. Shard Split Logic

- Convert 4 shards → 8 shards
- Virtual shards: easier than physical
- Calculate which users move where

5. Data Migration Worker

- Move users from old shard to new shard
- Batch by batch (don't lock)
- Verify data matches

6. Update Routing

- Once data moved, update shard directory
- Route requests to new shard
- Atomic update (no downtime)

7. Web Dashboard

- Show shard topology
- Show data distribution
- Show rebalancing progress
- Show hotspots

8. CLI Operations

```
$ shardmaster add-shard --region us-west
$ shardmaster rebalance --target-num-shards 8
$ shardmaster status
```



Week-by-Week

Week 1-2: Design shard directory + routing

Week 3-4: Implement hash sharding

Week 5-6: Build shard split algorithm

Week 7-8: Build data migration

Week 9-10: Routing updates + consistency

Week 11: Dashboard + monitoring

Week 12: Testing + demo



V1 Deliverables

- ☐ Shard directory service
 - ☐ Routing middleware
 - ☐ Hash-based sharding
 - ☐ Shard split (4→8 shards)
 - ☐ Data migration worker
 - ☐ Automatic routing update
 - ☐ Web dashboard
 - ☐ CLI for operations
 - ☐ Support 2-4 shards
 - ☐ Zero-downtime rebalancing
 - ☐ Demo video
-



GRADING (100 pts)

Functionality (40): Sharding works

Consistency (15): No data loss

Code Quality (20)

Deployment (10)

Docs (10)

Testing (5)

Passing: 70+

V2 OPTIONAL

- Consistent hashing
 - Hotspot detection + auto-rebalance
 - Multi-tenant isolation
 - Geographic sharding
 - Distributed transactions
-